

SmartFix — API Contract

Audience: Frontend (React) developer **Backend:** FastAPI (Python) **Status:** Draft v1 — see "Stability notes" at the end before assuming any field is locked.

1. Overview

The SmartFix backend exposes a small REST API with three concerns:

1. **Auth** — magic-link login, shared by all users (workers and admins).
2. **RAG query** — workers ask questions about a specific machine; the backend retrieves relevant manual excerpts and runs them through an LLM.
3. **Admin** — admins upload new machine PDFs; the backend parses, chunks, embeds, and indexes them in the background.

Roles: `worker` (read-only) and `admin` (read + manage machines). Role is determined server-side based on the user's email.

2. Conventions

- **Base URL:** `/` (FastAPI app root). When deployed, the frontend will configure this via an env var, e.g. `API_BASE=https://api.smartfix.tecdia.com`.
- **Content type:** `application/json` for all requests/responses unless noted (the only exception is `multipart/form-data` for PDF uploads).
- **Authentication:** session cookie set during `/auth/verify`. The cookie is `httpOnly`, `Secure`, `SameSite=Lax`. The frontend `fetch` calls **must include** `credentials: 'include'`.
- **Error envelope:** every non-2xx response uses this shape:

```
{ "detail": "human-readable message", "code": "machine_readable_code" }
```

- **Timestamps:** ISO 8601 UTC, e.g. `2026-04-29T10:01:23Z`.
 - **Unknown fields:** the frontend should ignore unknown response fields (so additive backend changes don't break the client).
-

3. Magic-link authentication flow

1. User opens app, sees email input
2. `POST /auth/request-link { email }`
 - └ Server: if email allowlisted → email a token link
3. User opens email, taps link
4. `GET /auth/verify?token=...`
 - └ Server: validates token, creates session, sets cookie
 - └ Server: 302 redirects to /
5. React app loads, calls `GET /auth/me`
 - └ Renders worker UI or admin panel based on `role`
6. Session lasts 30 days
 - └ Any 401 response → React routes back to login screen

Notes:

- The same flow is used by both workers and admins.
- The role is decided at step 4 by checking whether the email is in the server's `ADMIN_EMAILS` allowlist.
- Tokens expire 15 minutes after issuance, are single-use, and are invalidated after consumption.

4. Endpoint reference

4.1 Auth

`POST /auth/request-link`

Send a magic-link email.

- **Auth required:** No

Request

```
{ "email": "operator@tecdia.com.ph" }
```

Response — 200 (always, regardless of allowlist match)

```
{ "ok": true }
```

Response — 422 (malformed email)

```
{ "detail": "Invalid email format", "code": "invalid_email" }
```

The server intentionally returns 200 even for non-allowlisted emails, so attackers can't enumerate valid users. The actual email is only sent to allowlisted addresses.

`GET /auth/verify?token={token}`

Consume a magic-link token, create a session, set cookie, redirect to app.

- **Auth required:** No (this endpoint *creates* the session)

Query params

Name	Type	Required	Description
token	string	Yes	Token received in the email link.

Response — 302 redirect

- On success: `Set-Cookie: session=...; HttpOnly; Secure; SameSite=Lax; Max-Age=2592000` and redirect to `/`.
- On invalid/expired/used token: redirect to `/?login_error=expired`. The frontend reads the query param and shows an error.

The frontend doesn't call this directly. The user clicks the link in their email; the browser navigates to this endpoint. The React app only needs to handle the `?login_error=...` query param when it boots.

GET `/auth/me`

Return the currently logged-in user.

- **Auth required:** Yes

Response — 200

```
{
  "authenticated": true,
  "email": "operator@tecdia.com.ph",
  "role": "worker",
  "session_expires_at": "2026-05-29T10:01:23Z"
}
```

role is one of: "worker" | "admin".

Response — 401

```
{ "detail": "Not authenticated", "code": "unauthenticated" }
```

Frontend calls this on app load to render the right UI. On 401, route to login screen.

POST `/auth/logout`

Invalidate the current session.

- **Auth required:** Yes
- **Request body:** *(empty)*

Response — 200

```
{ "ok": true }
```

The server clears the cookie via `Set-Cookie: session=; Max-Age=0`.

4.2 Worker + Admin (any authenticated user)

GET /machines

List machines available to query against.

- **Auth required:** Yes (any role)

Response — 200

```
{
  "machines": [
    {
      "id": "INJECTION_MOLDING_MACHINE",
      "display_name": "Injection Molding Machine (IMM-750)",
      "chunk_count": 16
    },
    {
      "id": "LASER_CUTTING_MACHINE",
      "display_name": "Laser Cutting Machine (LC-2040)",
      "chunk_count": 11
    }
  ]
}
```

Use `id` as the value sent in `machine_filter` to `/query`. Use `display_name` in the UI dropdown.

POST /query

Ask a question about a specific machine's documentation.

- **Auth required:** Yes (any role)

Request

```
{
  "question": "What does error E-04 mean?",
  "machine_filter": "INJECTION_MOLDING_MACHINE",
  "history": [
```

```
{ "role": "user", "content": "What does error E-02 mean?" },
{ "role": "assistant", "content": "Error E-02 indicates ..." }
]
```

Field	Type	Required	Notes
question	string	Yes	Max length 500 chars.
machine_filter	string	Yes	Must match an <code>id</code> from <code>/machines</code> .
history	array	No	Last N turns. Server caps at 8 messages. Each item: <code>{role: "user" "assistant", content}</code> .

Response — 200

```
{
  "status": "success",
  "answer": "Error E-04 indicates the clamping force has not reached the target setpoint",
  "sources": [
    { "document": "INJECTION_MOLDING_MACHINE.pdf", "page": 5 },
    { "document": "INJECTION_MOLDING_MACHINE.pdf", "page": 6 }
  ]
}
```

`status` is one of:

Value	Meaning
success	<code>answer</code> and <code>sources</code> are populated.
not_found	No relevant chunks for this question/machine. <code>sources</code> is empty; <code>answer</code> is generic.
error	LLM service unavailable. <code>sources</code> is empty.

Response — 422 (validation, e.g. question too long)

```
{ "detail": "question must be ≤500 chars", "code": "validation_error" }
```

4.3 Admin only

All admin endpoints require `role: "admin"` on the session. A worker hitting these gets 403.

POST `/admin/machines`

Upload a new PDF for ingestion. Returns immediately with a `job_id`; ingestion runs in the background.

- **Auth required:** Yes (admin)
- **Content type:** `multipart/form-data`

Form fields

Field	Type	Required	Notes
file	file	Yes	PDF only. Max 50 MB.
machine_id	string	Yes	Slug, [A-Z0-9_]+. Must not already exist.
display_name	string	Yes	Human-readable, e.g. "Injection Molding Machine (IMM-750)".

Response — 202 Accepted

```
{ "job_id": "job_abc123", "status": "queued" }
```

Response — 409

```
{ "detail": "Machine already exists", "code": "machine_exists" }
```

Response — 413

```
{ "detail": "File exceeds 50 MB", "code": "file_too_large" }
```

GET /admin/jobs/{job_id}

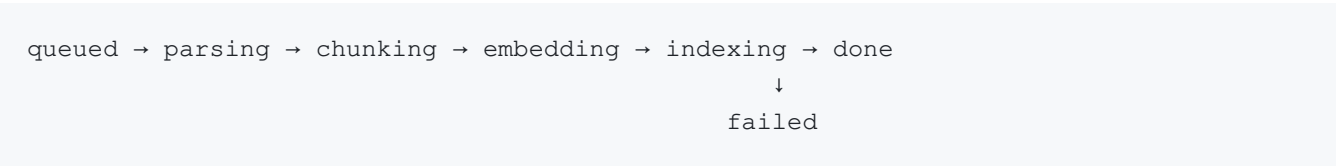
Poll an ingestion job's status.

- **Auth required:** Yes (admin)

Response — 200

```
{
  "job_id": "job_abc123",
  "machine_id": "NEW_MACHINE",
  "status": "embedding",
  "step": "Embedding 42 of 87 chunks",
  "progress": 0.48,
  "started_at": "2026-04-29T10:01:23Z",
  "finished_at": null,
  "error": null
}
```

Job status lifecycle



When `status === "failed"`, `error` contains the message to show the admin. When `status === "done"`, `finished_at` is set and the new machine appears in `/machines`.

Frontend should poll this every ~2 seconds while `status` is not `done` or `failed`.

GET `/admin/machines`

List machines with extra admin metadata.

- **Auth required:** Yes (admin)

Response — 200

```
{
  "machines": [
    {
      "id": "INJECTION_MOLDING_MACHINE",
      "display_name": "Injection Molding Machine (IMM-750)",
      "chunk_count": 16,
      "uploaded_at": "2026-04-15T08:21:00Z",
      "uploaded_by": "alice@tecdia.com.ph",
      "pdf_size_bytes": 2451200
    }
  ]
}
```

DELETE `/admin/machines/{machine_id}`

Remove a machine and all its chunks from the index.

- **Auth required:** Yes (admin)

Response — 200

```
{ "ok": true, "deleted_chunks": 16 }
```

Response — 404

```
{ "detail": "Machine not found", "code": "not_found" }
```

4.4 Health

GET `/health`

Liveness probe.

- **Auth required:** No

Response — 200

```
{ "status": "ok", "version": "0.1.0" }
```

5. Standard error codes

HTTP	code	When
400	bad_request	Malformed body
401	unauthenticated	No session, expired session, or invalid cookie
403	forbidden	Authenticated but role insufficient (worker hitting <code>/admin/*</code>)
404	not_found	Resource doesn't exist
409	machine_exists	Duplicate <code>machine_id</code> on upload
413	file_too_large	PDF over 50 MB
422	validation_error	Pydantic validation failed (per-field details in <code>detail</code>)
429	rate_limited	Reserved for future rate limiting
500	internal_error	Unhandled server error (with friendly generic message)

6. Recommended build order for the frontend

Each step is independently shippable.

1. **Chat UI against `/query` + `/machines`** with a hardcoded mock session (skip login). Most of the value is here.
2. **Magic-link login flow** — `/auth/request-link` form, `/auth/verify` redirect handler, `/auth/me` on app load.
3. **Admin panel** behind a `role === "admin"` gate. Upload form + progress polling + machine list/delete.

7. In-chat memory (15-minute idle expiry)

The frontend manages chat history client-side, so the backend stays stateless. Implementation guidance:

- Store `{history: Turn[], lastActivity: number}` in `localStorage` under key


```
smartfix.history.
```

- On app load, if `Date.now() - lastActivity > 15 * 60 * 1000`, clear the stored history.
- On every `/query` request, include the stored history in the request body.
- Append both the user turn and the assistant turn after each response, then update `lastActivity`.
- Provide a "Start over" button that clears the storage and the in-memory state.

This survives accidental page reload, persists across phone sleep cycles within the 15-minute window, and auto-clears after a shift change.

8. Stability notes

What's locked in (won't change)

- Auth flow shape.
- Role enum (`worker` | `admin`); future roles will be additive.
- Request fields on `/query` (`question`, `machine_filter`, `history`).
- Response status enum on `/query` (`success` | `not_found` | `error`).
- Async ingestion pattern (`POST` returns `job_id`; `POLL` `/admin/jobs/{id}`).
- Job status lifecycle.
- Error envelope shape (`{detail, code}`).

What may extend (additive — old code still works)

- `sources[].text` field on `/query` responses (for source previews).
- Display name / last login on `/auth/me`.
- Finer-grained progress on `/admin/jobs/{id}`.
- Machine grouping/categories on `/machines`.

What could shift if cloud topology changes

- **Cookies vs Bearer tokens:** if the deployed frontend ends up on a different domain than the API, the auth credential may switch from a cookie to a `Authorization: Bearer ...` header. To future-proof, hide auth credential handling behind a `getAuthHeaders() / credentials: 'include'` wrapper in the fetch layer.

What's open / pending Tecdia decisions

- **Vector DB choice** (Qdrant Cloud vs ChromaDB server). Internal to backend; no contract impact.
- **Email allowlist mechanism** (per-email vs domain restriction). No contract impact.
- **Email provider** (Resend vs Tecdia SMTP). No contract impact.

End of contract